Neurocomputing 367 (2019) 287–298

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

Global relative position space based pooling for fine-grained vehicle recognition

Ye Xiang, Ying Fu*, Hua Huang

School of Computer Science and Technology, Beijing Institute of Technology, Beijing, China

ARTICLE INFO

Article history:
Received 9 January 2019
Revised 8 May 2019
Accepted 27 July 2019
Available online 9 August 2019

Communicated by Zidong Wang

Keywords:
Fine-grained vehicle recognition
Global relative position space based pooling (GRPSP)
Convolution neural network (CNN)

ABSTRACT

In the field of fine-grained vehicle recognition, large intra-class variation often affects the final prediction seriously. One of the principal reasons is unstable change of absolute positions for discriminative features, which can not be completely solved by the region of interest (ROI) localization for inevitable deviations. In this paper, we propose a global relative position space based pooling (GRPSP) method to replace the absolute position information with global relative position information, by which the discriminative features can be well aligned and thereby the intra-class variation is reduced effectively. Specifically, the discriminative features containing both absolute position information and appearance information are extracted by a convolution neural network (CNN). We then transform the absolute positions into global relative positions, and estimate position probabilities of locating at discrete representatives of global relative position space. Accordingly, the joint probabilities of different kinds of features locating at different representative positions can be calculated, and constitute the pooling vector. Lastly, the Support Vector Machine (SVM) is trained for classification. Experimental results on public and self-building challenging datasets both show that our proposed method can improve various CNNs and is the state-of-the-art.

1. Introduction

In recent years, fine-grained vehicle recognition has attracted researchers' much attention in intelligent transportation systems (ITS) field. The fine-grained vehicle types not only refer to a variety of manufactures and models, but also concretely specify the years and versions as long as the vehicle appearances are different. In spite of that, there exists a very related issue of vehicle type classification, which merely distinguishes the vehicles between sedan, minivan, SUV, truck, and so on. By contrast, the fine-grained vehicle recognition can offer more comprehensive assistance in many situations. For example, fine-grained information can be used to specifically identify cars for catching criminals when the license plates are obscured or non-existent. Also, in public service, parking lots can automatically charge and reasonably allocate space through fine-grained vehicle categories. Furthermore, detailed vehicle models can assist in analyzing traffic flow, and thereby help make strategies for traffic control [1].

Many methods have been presented for the fine-grained vehicle recognition [2]. According to the image view, they can be mainly classified into three categories, i.e. front-view based method [3,4], rear-view based method [5], and side-view based method [6]. Most of these methods belong to the first category and are easy to reach better results, due to the more discriminative features in frontal view vehicle. Among public datasets of frontal view vehicles, surveillance data is more common than web data, while web data has more intra-class variations and is more difficult for recognition. Hence we pay more attentions on web data and collect a challenging dataset for it. In addition, based on the feature level, all methods can also be divided into three classes, i.e. low-level features based method [7–11], mid-level features based method [12], and high-level features based method [13,14]. Among those different features, the deep convolutional features extracted by CNN can be regarded as the high-level features, and have drawn much attentions for their excellent performance. Besides, by the aid of pre-training on a large dataset, the model of CNN can obtain even better result, which has been shown in many visual related works [15–17]. Hence, some researchers [18,19] have employed this technique on fine-grained vehicle recognition and gained great benefits.

All of these related works have faced identical and severe challenges, which are resulted from small inter-class variations and large intra-class variations. The reason is that a variety of vehicle models sharing the inherent parts have the similar appearances, yet images coming from the same model vary greatly as the illumination or angle of view changes. Especially when the angle

* Corresponding author.
E-mail addresses: xiangye@bit.edu.cn (Y. Xiang), fuying@bit.edu.cn (Y. Fu), huahuang@bit.edu.cn (H. Huang).

https://doi.org/10.1016/j.neucom.2019.07.098
0925-2312/

of pitch differs, some discriminative vehicle parts may obviously move their positions, resulting in large intra-class variations. This problem can not be completely solved by just the region of interest (ROI) localization for inevitable deviations.

Therefore, to reduce intra-class variations, position information should be exploited reasonably. In past studies, spatial pyramid matching (SPM) [20,21] was most widely adopted and has been proved very effective. In addition, on the basis of CNN, Fang et al. [22] proposed a coarse-to-fine method to take fully advantage of the position information for fine-grained vehicle model recognition by detecting discriminative regions automatically. Conversely, in [7], Siddiqui et al. adopted a bag of speeded-up robust features (SURF) and built global representation of whole image without any position information. This directly caused the poor performance.

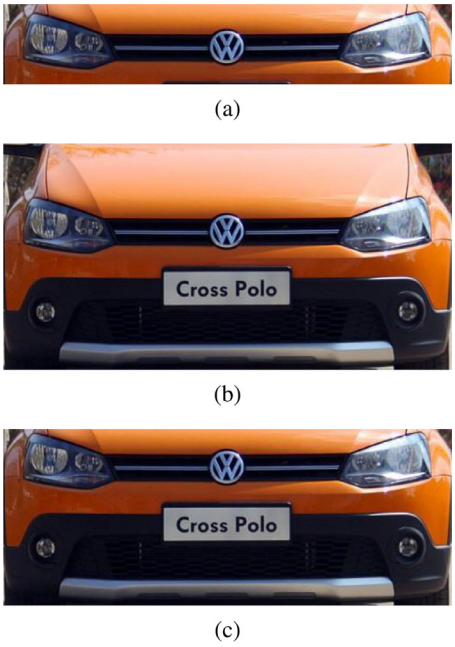
Although most of these methods have explored position information, the positions they used are absolute rather than relative. For instance, in SPM process, the input images are divided into several pooling regions at fixed absolute coordinates. Also, in CNN based classification problem, one of the major ways to create final image representations, is applying the pre-trained CNN to certain subregions of input images with fixed absolute positions, and aggregating the activations from those fixed regions together [23]. It is undeniable that the absolute position information is sometimes sufficient for task. However, for fine-grained vehicle recognition, the position varies a lot especially when the angle of pitch differs, which leads to the misalignment of discriminative features. At this circumstance, absolute position information is ill-suited and relative position information is recommended.

Besides, the approach of integrating CNN and SVM [24] has been widely used. It replaces the last fully-connected layer of CNN for classification with a SVM classifier, whose reasonability in theories has been explained [25]. Specifically, CNN can be regarded as an extension model of multi-layer perceptron (MLP), which attempts to minimize the empirical errors in training set. Conversely, SVM classifier aims to minimize the generalization errors on new data. In consequence, the generalization ability of SVM is superior than CNN. From another point of view, the layer of MLP for classification tends to assign a high value (nearly $+1$) to one neuron and a low value (nearly -1) to all other neurons, while SVM classifier estimates the probabilities of decisions. Therefore, SVM also has better reliability.

In this paper, we mainly deal with the challenges of "Ambiguity" and "Heterogeneous Views" (pitch angle) from Boukerche et al. [2] in fine-grained vehicle recognition of frontal view. To this end, we propose a novel pooling method based on the global relative positions for high-level features, we call it as global relative position space based pooling (GRPSP). The high-level features are obtained by extracting the feature maps output by a certain layer of a fine-tuned CNN. Then using the extracted features with both absolute position information and appearance information, we calculate a vector of joint probabilities that imply different kinds of features locating at different global relative positions. This vector with fixed length is just our pooling vector which can serve as the final representation of an input image. Lastly, we adopt the linear SVM to train a classifier. The classification results in experiments have validated the effectiveness of our method.

The main contributions of this paper can be summarized as follows:

- We propose a novel method for fine-grained vehicle recognition using the global relative position information of discriminative features. It contributes greatly to alignment of features by global relative position space, and thereby effectively reduces intra-class variations.
- We present a new pooling method, in which the length of final representation vector is fixed and independent of input image size. It is important when large scale image with detail information is needed. This pooling method can also be used in many other recognition fields.
- We collect a challenging dataset which owns great intra-class variations and very fine-grained categories for vehicle recognition. It contains 4000 frontal view vehicle images from web, splitting into 400 fine-grained classes.



(a)

(b)

(c)

Fig. 1. Three different types of ROIs. They are all head parts of vehicle. (a) is the head part without hood and licence plate, (b) is the head part with hood and licence plate, and (c) is the head part without hood but with licence plate.

The remainder of this paper is organized as follows. In Section 2, we provide an overview of the related works for vehicle recognition. The detailed procedure of GRPSP based recognition method is then presented in Section 3. Section 4 is about experiments, in which we introduce our different models by combining with different CNNs, specific experimental setup, effects of different parameters, and comparison results on two datasets. Lastly, conclusions are provided in Section 5.

2. Related work

Fine-grained vehicle recognition has been widely studied. In most cases, it includes three essential steps, which are ROI extraction, feature extraction and representation, and classification, respectively. Here, we review related work based on these three steps.

2.1. ROI extraction

Since our method focuses on frontal view fine-grained vehicle recognition, we mainly investigate ROI extraction methods for frontal view vehicle. These methods can be classified into three categories. Methods in first category crop images manually, such as [26]. Methods in second category seek the boundary line of ROI by reference to detected licence plate [12,27–29]. Methods in third category rely on calculation of symmetrical feature points, for instance, symmetrical SURF points were used for locating ROI in [8,9]. The ROIs obtained from various methods are illustrated in Fig. 1. They are the head part without hood and licence plate, head part with hood and licence plate, and head part without hood

but with licence plate, respectively. Among these ROIs, the third type is most widely used. Since the region around licence plate often contains important information, the first type ROI is not recommended. The second type is unpopular too, because the hood region contains not only little information for fine-grained vehicle recognition, but also a diversity of noises caused by reflections, stickers, and so on.

All of the ROI extraction methods are for alignment of features, but it can be observed from Fig. 1 that the boundary line can not be defined accurately even by human eyes. This means deviations which may disturb following recognition performance seriously are inevitable. Therefore, in our method, the whole image instead of ROI is used for input, and alignment of features is done by the proposed pooling algorithm.

2.2. Feature extraction and representation

Taking the input image, features at varied levels can be extracted. Among them, the low-level features are mostly used, e.g. scale-invariant feature transform (SIFT) [10,11], SURF [7–9], Histogram of Oriented Gradients (HOG) [30], etc. On the foundation of these single type features, combination of features from multiple types is proposed. In [26], pyramid histogram of oriented gradients (PHOG) and Curvelet features were normalized and jointed together for vehicle make classification. In [31], Zhang adopted the mixture of Gabor and PHOG to classify vehicle types. As the feature level goes up, mid-level features are explored. Fraz et al. [12] put forward a novel mid-level feature representation to capture the fine-grained information in discriminative patches of vehicle. Tang et al. [32] also explored the patch-level features for weakly supervised object classification. In addition, the higher level features are further studied. For example, the CNN, which was shown to possess the ability of representing high-level semantic information in [33], has been employed for vehicle recognition in [13,14,34], etc.

After feature extraction, the encoding procedure is mostly indispensable for traditional handcrafted features. The most simple encoding method called vector quantization (VQ) was used in [7] to build the dictionary of SURF features. This method assigned each feature to its closest codeword, and therefore led to a large quantization loss. An improved effective encoding method called Fisher Vector (FV) was proposed in [35], which described the signal with a gradient vector and was adopted to compute the visual words of patches in [12]. The FV was also explored for constructing the end-to-end trainable system by combining with CNN [36]. Besides, there is another popular encoding method called locality-constrained linear coding (LLC) [37]. The LLC method utilized locality constraint to achieve the sparsity of codes, and was applied in [38]. Both FV and LLC can obtain satisfactory result even combing with simple linear SVM classifier. However, LLC is usually more fast and produces vectors with intuitive and straightforward physical meaning. Hence, we adopt LLC in our method.

Based upon the obtained codings or extracted features, different pooling approaches are chosen to constitute the final representation vector. Concretely, Siddiqui et al. [7] took the simplest pooling approach which just concatenated the encoded features of whole input image into a vector. In [39], the most common pooling approach, namely SPM, was selected. This approach divided input image into subregions with a pyramid way, and the operation of max-pooling, sum-pooling or average-pooling was employed to mix the multiple feature codes in each subregion. Besides, a learnable pooling function was proposed in [40], which was applied on bag of shape features for shape recognition. These various pooling methods are respectively adaptive to different situations or tasks. In our paper for fine-grained recognition, since the large intra-class variance is more concerned, we mainly study the pooling method that can fully utilize the position information to effectively align discriminative features.

2.3. Classification

After obtaining the final representation vectors for training and testing images, a classification strategy is applied to make the prediction. SVM is one of the most generally accepted classifiers. It has been widely utilized in vehicle make and model recognition field [7,9]. The vehicle make and model recognition belongs to the multi-class classification problem. To solve it, the one-versus-all strategy is usually employed [22]. Besides, Huang et al. [27] simply used the nearest-neighbor classifier to determine vehicle type of testing image. Zhang and Zhao [26] exploited the MLP ensemble for achieving a reasonable classification result by taking advantage of the diversity among MLP components. Furthermore, the CNN architecture often takes fully-connected layer combined with soft-max function for classification [6].

The combination of multiple classifiers is also explored. For example, Zhang [31] utilized four types of classifiers, which were k-nearest neighbors (kNNs), MLPs, SVMs, and random forests. Boonsim and Prakoonwit [41] adopted three types of classifiers, including SVM, decision tree, and kNNs. Another special example for combination of classifiers was based on the hierarchical relationship among fine-grained vehicle labels, which was used in [42]. Through majority voting of different classifiers with rich diversity, the high reliability of decisions can be guaranteed. However, considering the expensive computation cost of multiple classifiers, we just select the common SVM as our classifier.

3. The proposed method

The framework of our method is illustrated in Fig. 2. It takes the whole image of front-view vehicle as input. The integral recognition framework mainly includes three steps, which are feature extraction, global relative position space based pooling, and classifier learning. Here, we describe these three steps, respectively.

3.1. Feature extraction

We choose CNN to extract the features as it has obtained great success in many computer vision fields, such as object detection [15], classification [43], and face verification [44,45]. A CNN usually consists of convolution layers, pooling layers, and fully-connected layers. In most layers, there exist parameters that are used to connect the input neuron units to output neuron units. The responses of output neuron units from each layer constitute a group of feature maps, which represents a specific level of feature representation. As the layer gets deeper, output feature maps become smaller and more discriminative. All feature maps of different sizes serving as extracted features form the hierarchical representation of input signal.

There are many kinds of CNNs. Almost all commonly used CNNs for image classification can be employed and combined with our proposed pooling method. Among these CNNs, CaffeNet [46] which is the variant of AlexNet [43] only contains five convolution layers, but has been proved to be effective in many fields [17,22]. Other types of deep CNNs, such as GoogLeNet [47], VGG [48], and ResNet [49], usually include much more convolution layers and can perform better than CaffeNet, especially for complex and challenging problem. We will pick out an appropriate CNN via experiments for the issue we focus on.

From different layers of a CNN, we need to select one layer and take its output feature maps as extracted features. Normally, the posterior layers are more discriminative than the preceding layers, and this will be validated in Section 4.3.1. Besides, the features

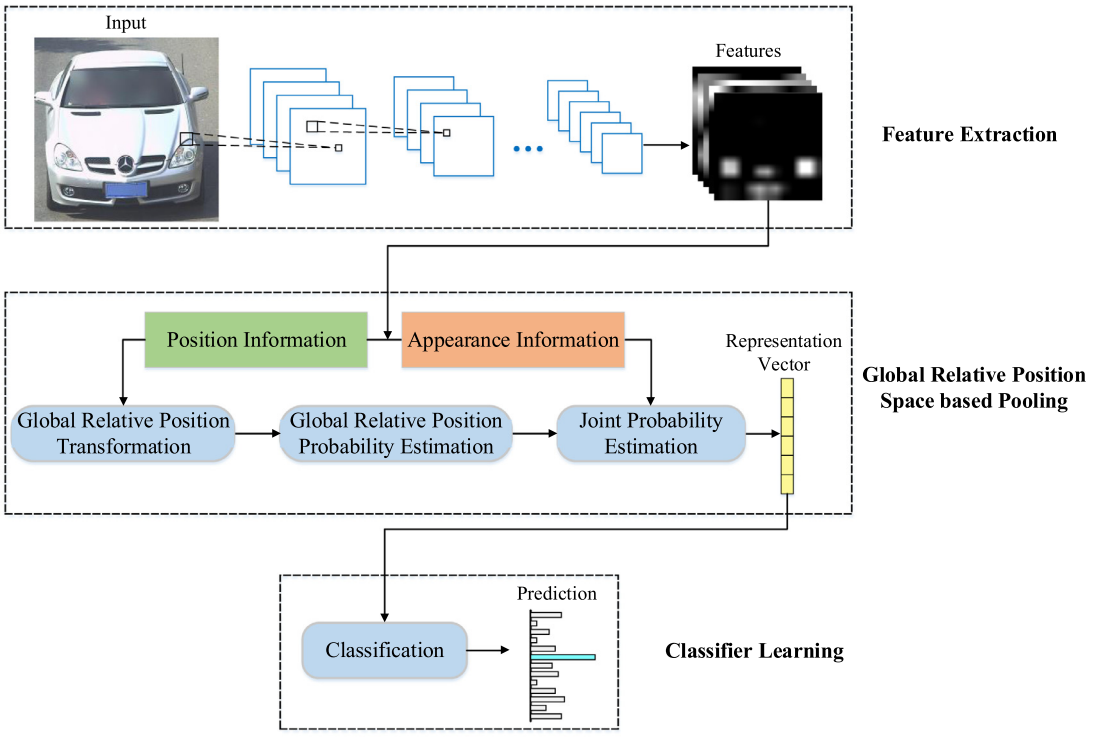


Fig. 2. The framework of the proposed fine-grained vehicle recognition method. It uses the whole image of front-view vehicle as input. The recognition procedure contains three steps. Feature extraction is the first step, which generates the deep convolutional features using CNN. Global relative position space based pooling is the second step. It estimates the joint probability of position and appearance information to represent features extracted from the first step. Classifier learning is the last step to perform classification with representation vector which is obtained from pooling algorithm.

from fully-connected layers are insufficient for capturing spatial information. Therefore, we employ feature maps from the last max-pooling (convolution for GoogLeNet and ResNet) layer before fully-connected layer as the extracted features.

The extracted features are 3D, in which each 2D feature map stands for the response of one kind of feature defined by a fixed convolution kernel. The response includes both absolute position information (i.e. pixel coordinate) and appearance information (i.e. pixel intensity) of feature.

In addition, it is remarkable that some researchers tend to enlarge input image of CNN to get bigger feature maps with more detail information [50,51]. This trick is helpful especially for fine-grained recognition. However, along with supplement of detail information, a problem of more unstable position shift of salient features occurs. At this circumstance, our proposed relative position based method which can just perfectly solve this problem shows great advantage.

3.2. Global relative position space based pooling

Based on extracted multiple kinds of features with explicit positions, pooling algorithm aims to represent the input image with a vector of fixed length to better distinguish different categories. In our GRPSP method, we propose to adopt the probabilities of different kinds of features locating at representative global relative positions to constitute the pooling vector, i.e.,

$$\boldsymbol{f} = [\boldsymbol{v}_1^{\mathrm{T}}, \ldots, \boldsymbol{v}_m^{\mathrm{T}}, \ldots, \boldsymbol{v}_M^{\mathrm{T}}]^{\mathrm{T}}, \tag{1}$$

in which $\boldsymbol{f}$ is the pooling vector, $\boldsymbol{v}_m$ is the sub-vector that contains probability values of the mth kind of features locating at various discrete representative positions, and M is the total number of features, i.e., channel number of feature maps. With N to denote the number of discrete representative positions in global relative position space, we can get $\boldsymbol{v}_m \in \mathbb{R}^{N\times 1}$ and $\boldsymbol{f} \in \mathbb{R}^{MN\times 1}$.

To obtain the $\boldsymbol{v}_m$ and $\boldsymbol{f}$, we find from their definitions that the joint probability of feature appearance and feature position should be computed. The feature appearance probability is intuitively proportional to the pixel intensity of feature maps. Hence we directly use the pixel intensity as the feature appearance probability for simplification. With regard to the feature position probability, we take two steps for computation, which are the global relative position transformation and position probability estimation, respectively. During the first step, we transform the absolute position of each feature point into global relative position, which contains interaction information between different feature points and is thus more robust than absolute position. During the second step, we first make discretization and find representatives for continuous global relative position space, then estimate the probabilities of each global relative position belonging to the pre-computed discrete representatives. The entire pooling process is illustrated in Algorithm 1.

3.2.1. Global relative position transformation

To transform the absolute position of feature point into global relative position, we need to build a 2D saliency map $\mathbf{S}$, on which the saliency of each absolute position can help guide the interaction between feature points of different positions.

We compute the saliency degree of each absolute position for saliency map $\mathbf{S}$ by considering all kinds of features. Since each extracted feature map only corresponds to one kind of features, we decide to adopt a simple pooling algorithm to unite all feature maps together. There are two frequently-used ways that can be chosen from, i.e., max-pooling and average-pooling along channel dimension. We use the channel-wise average-pooling like [22]. Therefore, the 2D saliency map $\mathbf{S}$ is calculated as:

$$\mathbf{S} = \frac{1}{M}\sum_{m=1}^{M}\mathbf{G}_m, \tag{2}$$

Algorithm 1 Global Relative Position Space based Pooling.

Input: extracted feature maps **G** from a given image
Output: pooling vector $\boldsymbol{f}$

```
compute saliency map S using Eq. (2)
for each absolute position vector x_k in S do
    transform into global relative position vector r_k using Eq. (7)
end for
find representatives B for global relative position space by Kmeans++ clustering
for each r_k in S do
    encode into e_k which is a vector comprised of position probabilities through Eq. (8)
end for
compute f which is comprised of joint probabilities through Eqns. (10), (11), and (1)
```

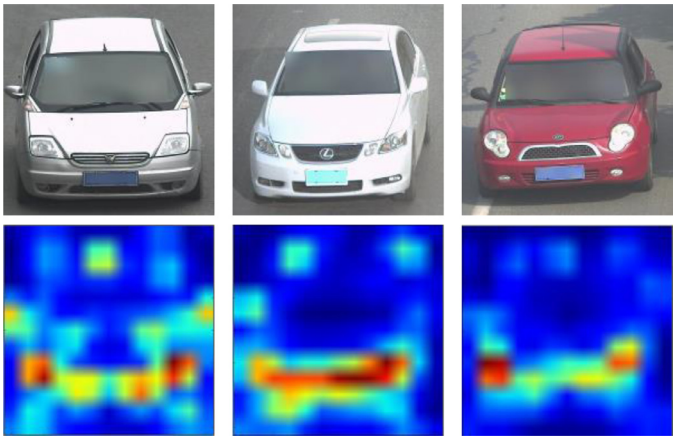


Fig. 3. Sample saliency maps generated from vehicle images. The first row corresponds to input images, and the second row corresponds to their saliency maps.

in which $\mathbf{G}_m$ represents the mth feature map. By using $\mathbf{G} \in \mathbb{R}^{M\times L\times L}$, we can get $\mathbf{S} \in \mathbb{R}^{L\times L}$. We illustrate several sample saliency maps generated from vehicle images in Fig. 3.

After obtaining the 2D saliency map **S**, we can transform the absolute position into global relative position by interaction between salient feature points on **S**.

For each feature point in saliency map **S**, its global relative position can be computed according to the distribution of other feature points relative to it. Specifically, we sum all these relative vectors together and multiply by -1. This can be written as:

$$\boldsymbol{r}_k = -\sum_{k'=1,k'\neq k}^{K} (\boldsymbol{x}_{k'} - \boldsymbol{x}_k), \tag{3}$$

in which k and k' are both the indexes of feature points in **S**, K is the total number of discrete absolute positions for all feature points, i.e., $K = L^2$, $\boldsymbol{x}_k$ and $\boldsymbol{r}_k$ are respectively the absolute position vector and global relative position vector to be calculated for feature point in **S**.

Furthermore, the calculation of global relative position vector should also be influenced by the feature intensity, i.e., the saliency degree of feature point in **S**. For each feature point involved in the calculation of global relative position vector, the higher saliency degree, the lower probability of being noise. Accordingly, we regard the pixel intensities of feature points in saliency map **S** as weights, and rewrite the Eq. (3) into:

$$\boldsymbol{r}_k = -\sum_{k'=1,k'\neq k}^{K} s_{k'}(\boldsymbol{x}_{k'} - \boldsymbol{x}_k). \tag{4}$$

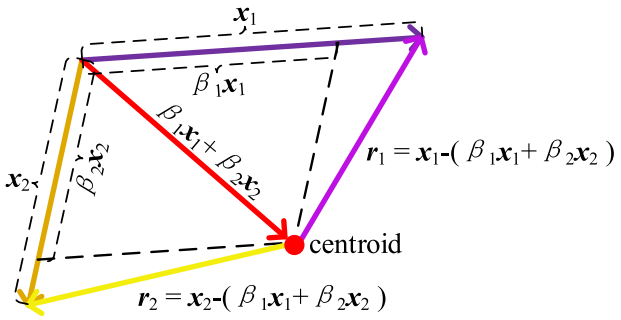


Fig. 4. Transformation from absolute position vectors into global relative position vectors. The $K = 2$ features points are utilized for clarity. $\boldsymbol{x}$ represents the absolute position vector, $\boldsymbol{r}$ stands for the global relative position vector, and β means the weight value.

Here $s_{k'}$ stands for the weight value of the k'th feature point. All of the weights are normalized by forcing their sum to be 1, thus we obtain

$$\boldsymbol{r}_k = -\sum_{k'=1,k'\neq k}^{K} \beta_{k'}(\boldsymbol{x}_{k'} - \boldsymbol{x}_k), \tag{5}$$

in which $\beta_{k'}$ is the normalized weight, i.e.

$$\beta_{k'} = \frac{s_{k'}}{\sum_{k=1}^{K} s_k}. \tag{6}$$

We further analyze and simplify the Eq. (5) as following:

$$\begin{aligned}
\boldsymbol{r}_k &= -\left(\sum_{k'=1,k'\neq k}^{K} \beta_{k'}\boldsymbol{x}_{k'} - \sum_{k'=1,k'\neq k}^{K} \beta_{k'}\boldsymbol{x}_k\right) \\
&= -\left(\sum_{k'=1}^{K} \beta_{k'}\boldsymbol{x}_{k'} - \sum_{k'=1}^{K} \beta_{k'}\boldsymbol{x}_k\right) \\
&= -\left(\sum_{k'=1}^{K} \beta_{k'}\boldsymbol{x}_{k'} - \boldsymbol{x}_k\right) \\
&= \boldsymbol{x}_k - \sum_{k'=1}^{K} \beta_{k'}\boldsymbol{x}_{k'}.
\end{aligned} \tag{7}$$

In Eq. (7), $\sum_{k'=1}^{K} \beta_{k'}\boldsymbol{x}_{k'}$ can be interpreted as the weighted centroid. It allows the pre-computation and keeps constant for all feature points of an input image, which thereby greatly saves the run time. We illustrate the transformation process with Fig. 4, in which $K = 2$ is used for example.

All in all, by applying the simplified transformation function derived in Eq. (7) to all feature points in saliency map **S**, we transform the absolute positions into global relative positions.

3.2.2. Global relative position probability estimation

Various global relative position vectors from all training images constitute the continuous global relative position space. Within this space, we first make discretization and find fixed representatives, then take the effect of aligning with different representatives as the probabilities of each global relative position vector to be estimated.

We adopt the unsupervised clustering technique to make discretization of global relative position space. The cluster centers are namely the fixed representatives, and the cluster number equals to the discrete representative position number, i.e., N. There are many kinds of unsupervised clustering techniques. We employ the Kmeans++ introduced in [52], which is shown to be fast and effective. The codebook obtained by Kmeans++ clustering is written as

B, in which each column is a codeword indicating one representative global relative position.

The obtained discrete representatives in **B** are sufficient for representing the whole global relative position space. Hence we estimate the probabilities of each global relative position vector being aligned with fixed representatives to keep the robustness. This kind of probability estimation can be formulated as an encoding problem, in which the codebook denoting the set of representative positions is given and the codes are just the required probabilities. We take the classical LLC algorithm [37] for encoding. It emphasizes the linearity and locality which are both very important for effective reconstruction. As a result, the probabilities of global relative position vector being aligned with different representatives are calculated by solving the following optimization function:

$$\arg\min_{\boldsymbol{e}_k} \|\boldsymbol{r}_k - \mathbf{B}\boldsymbol{e}_k\|^2 + \lambda\|\boldsymbol{d}_k \odot \boldsymbol{e}_k\|^2 \tag{8}$$
$$s.t.\ \mathbf{1}^{\mathrm{T}}\boldsymbol{e}_k = 1,$$

in which $\boldsymbol{e}_k$ is the encoding result of $\boldsymbol{r}_k$, i.e., a vector containing probability values, $\odot$ is the element-wise multiplication, and $\boldsymbol{d}_k$ denotes the locality adaptor that includes the values proportional to the similarities between codewords in **B** and input vector $\boldsymbol{r}_k$. $\boldsymbol{d}_k$ is defined as following:

$$\boldsymbol{d}_k = \exp\left(\frac{dist(\boldsymbol{r}_k, \mathbf{B})}{\sigma}\right), \tag{9}$$

in which $dist(\boldsymbol{r}_k, \mathbf{B}) = [\|\boldsymbol{r}_k - \boldsymbol{b}_1\|, \ldots, \|\boldsymbol{r}_k - \boldsymbol{b}_n\|, \ldots, \|\boldsymbol{r}_k - \boldsymbol{b}_N\|]^{\mathrm{T}}$, and is a vector containing Euclidean distances between $\boldsymbol{r}_k$ and codewords in **B**.

To sum up, we take advantage of the traditional clustering and encoding techniques to achieve the space discretization and probability estimation of global relative position.

3.2.3. Joint probability estimation

Based on the probability of feature position, the joint probability can then be estimated. Concretely, on one feature map, we can obtain a different joint probability vector for each pixel, but with the same meaning, which refers to the probabilities of current kind of features locating at different representative positions. The formula we adopt is

$$\boldsymbol{p}_{mk} = g_{mk}\boldsymbol{e}_k, \tag{10}$$

in which g_{mk} denotes the appearance probability value for the kth pixel in the mth feature map, and $\boldsymbol{p}_{mk}$ is the joint probability vector with length of N.

It can be seen from the Eq. (10) that there are multiple different joint probability vectors for the same feature map. However, what we need is a unique and comprehensive joint probability vector for one same feature map, e.g., $\boldsymbol{v}_m$. Hence we synthesize the different joint probability vectors for one same feature map, i.e., $\boldsymbol{p}_{mk}$ with k ranging from 1 to K. A common way of maximization for each dimension of vectors is utilized:

$$\boldsymbol{v}_m = \max_k \boldsymbol{p}_{mk}. \tag{11}$$

On the basis of $\boldsymbol{p}_{mk}$ and $\boldsymbol{v}_m$, the vector $\boldsymbol{f}$ in Eq. (1), which denotes the probabilities of different kinds of features locating at all different representative global relative positions, can be estimated. The final result of pooling vector will be directly used for following classifier learning.

3.3. Classifier learning

Various kinds of classifiers can be applied to train pooling vectors. We choose the linear SVM classifier which is efficient and the most capable of reflecting discriminability of vectors.

During SVM classifier training procedure for fine-grained recognition, we face a multi-class classification problem, which can be solved by the one-vs-all strategy. That means for any class, we train a linear two-category SVM classifier. The optimization function is

$$\min_{\boldsymbol{w},\eta,\xi} \frac{1}{2}\boldsymbol{w}^{\mathrm{T}}\boldsymbol{w} + \mu\sum_{j=1}^{J}\xi^{(j)} \tag{12}$$
$$s.t.\ y^{(j)}(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{f}^{(j)} + \eta) \geq 1 - \xi^{(j)},$$
$$\xi^{(j)} \geq 0,\ \ j = 1, \ldots, J,$$

in which $\boldsymbol{f}^{(j)}$ is the jth pooling vector, $y^{(j)} \in \{1, -1\}$ is the indicator of class label, $\mu > 0$ is the regularization parameter, and ξ is slack variable. After we finish the training of classifier, an optimal separating hyperplane determined by weight vector $\boldsymbol{w}$ and bias η can be discovered.

In testing phase, every one-vs-all SVM classifier outputs a score to imply the probability of input example belonging to the current positive class. Different scores correspond to different class labels, from which we pick the one that relates with the highest score as prediction.

4. Experiments

In this section, we combine different CNNs with our proposed method respectively, and use them for experiments. The experiments are conducted on two datasets, and the results are measured with two metrics. Using this experimental setup, we first adjust the parameters, then compare our proposed method with other state-of-the-art methods.

4.1. Models with different CNNs

We consider four kinds of commonly used CNNs: CaffeNet, GoogLeNet, VGG, and ResNet. They are separately combined with our proposed GRPSP method.

4.1.1. CaffeNet with GRPSP

CaffeNet only has five convolution layers, three max-pooling layers, and three fully-connected layers. We truncate it at the last max-pooling layer called *pool5* to extract features. If we input image with size of $227 \times 227 \times 3$ as required by the original CNN model, the size of our extracted feature maps is $6 \times 6 \times 256$. As we enlarge input image, the size of extracted feature maps also increases accordingly. Then features are further employed for global relative position space based pooling and SVM classification. We refer to this model as CaffeNet-GRPSP.

4.1.2. GoogLeNet with GRPSP

GoogLeNet is a deep and wide CNN. It has nine inception modules, in each of which there are six convolution layers. We truncate it at the output of the last inception module called *inception_5b/output* for feature extraction. If we input image with size of $224 \times 224 \times 3$ as required by the original CNN model, feature maps with size of $7 \times 7 \times 1024$ can be obtained. These feature maps are then used for pooling and classification. We refer to this model as GoogLeNet-GRPSP.

4.1.3. VGG16 with GRPSP

VGG is also a deep CNN. Its architecture can have different depths, including 11, 13, 16, and 19 layers. We decide to adopt configuration of 16 layers called VGG16 that can achieve outstanding performance with comparably low computation cost. VGG16 model is truncated at the last max-pooling layer called *pool5* for our feature extraction. The extracted feature maps have size of $7 \times 7 \times 512$

Fig. 5. Examples of CompCars dataset showing the rich diversity of lightings, weather conditions and inclinations. The horizontal center line (drawn in orange) serves as a reference to especially show the obvious vertical shift of vehicle parts.

Fig. 6. Examples of VMCLCars dataset exhibiting the great variations from lightings, elevations and inclinations. The horizontal center line (drawn in orange) acts as a reference to show the vertical shift of vehicle parts which is especially obvious and large.

Fig. 7. Examples of different vehicle models in VMCLCars dataset showing the fine-grained level. The first three columns correspond to different models of Audi A1 and the last three columns correspond to different models of Audi A3. The first row called Part1 is the headlight. The second row called Part2 locates at the lower-right corner of vehicle. The third row called Part3 is in the middle of head part, containing the vehicle logo.

when we take image with size of $224 \times 224 \times 3$ as input following original VGG16 model. Extracted feature maps are further utilized for pooling and classification to accomplish vehicle recognition. We refer to this model as VGG16-GRPSP.

4.1.4. ResNet101 with GRPSP

ResNet belongs to deep CNN too. Its architecture also has different depth, such as 18, 34, 50, 101, 152, etc. We choose configuration of 101 layers called ResNet101 for trade-off between recognition accuracy and computation cost. ResNet101 model is truncated at the output of last convolution layer called *res5c_relu* for feature extraction. Layer of *res5c_relu* produces feature maps with size of $7 \times 7 \times 2048$ when input image with size of $224 \times 224 \times 3$ is used as required by original ResNet101 model. The obtained feature maps are further employed for pooling and classification. We refer to this model as ResNet101-GRPSP.

4.2. Setup

4.2.1. Datasets

We conduct experiments on two datasets, including a public dataset and a self-building dataset. They are both captured from the frontal view.

The public CompCars dataset [18] has two kinds of data, which are surveillance-nature data and web-nature data. We pick surveillance-nature data, since the web-nature data contains vehicle of hybrid view which is the limitation of our method. The surveillance-nature data is collected by surveillance cameras. It possesses rich diversity of lightings, weather conditions and inclinations, which can be seen in Fig. 5. In total, there are 281 fine-grained vehicle models and 44,481 images for training and testing. The proportion of training set to testing set is 31148:13333, which is predesigned and adopted widely.

The other dataset built by our team is collected from web, and the vehicle types are all familiar to the public. We call it VMCLCars (https://github.com/ye-xiang/VMCLCars). The VMCLCars dataset has great variations from lightings, elevations and inclinations. Especially for vertical shift of salient part, such as vehicle headlight and logo, VMCLCars varies more largely than CompCars, which can be seen in Fig. 6. Then we divide the whole image set into fine-grained classes as long as the discriminative parts are different. For example, the small distinctions in vehicle subregions could identify three different models of Audi A1 and three different models of Audi A3, which are exhibited in Fig. 7 and exactly explain the fine-grained level of VMCLCars. This dataset includes 400 fine-grained vehicle models. Each model has 10 image samples, and the ratio between training set and testing set is 7:3 which is in accordance with CompCars dataset. The pre-determined split, especially for self-building dataset, may be biased. Hence we randomly split this dataset for five times, and the results are calculated by averaging over these splits. To sum up, comparing with CompCars, the variation is larger, the number of classes is more,

Table 1
The average accuracy and overall accuracy on VMCLCars dataset with feature extraction from different layers of CaffeNet.

Layer	Conv1	Pool1	Conv2	Pool2	Conv3	Conv4	Conv5	Pool5
Avr_Accuracy	78.49%	89.82%	92.69%	94.55%	94.78%	94.75%	95.49%	**96.37%**
Ovr_Accuracy	79.02%	90.33%	93.13%	94.89%	95.12%	95.10%	95.88%	**96.75%**

and the training samples are much less. All of these conditions directly lead to more serious challenges for vehicle recognition.

4.2.2. Metrics

In our experiments, two kinds of metrics are employed to evaluate different methods. We find that the average accuracy and overall accuracy are both commonly used measurement criteria for validating performance in many works about image recognition. For fine-grained vehicle recognition, since various categories may have widely different sample numbers, the average accuracy is actually more meaningful. We calculate the average accuracy and overall accuracy as following:

$$avr\_accuracy = \frac{\sum_{q=1}^{Q} \alpha_q}{Q}$$

$$ovr\_accuracy = \frac{\sum_{q=1}^{Q} \alpha_q z_q}{\sum_{q=1}^{Q} z_q}, \tag{13}$$

in which q stands for the category index changing from 1 to Q, α_q means the accuracy for the qth category, and z_q is the number of testing images for the qth category.

4.2.3. Implementation details

To extract feature maps for recognizing vehicles, we fine-tune the CNNs described in Section 4.1. The parameters in each CNN are initialized by the same CNN model that is pre-trained on a large-scale dataset, namely ImageNet dataset [53]. The total number of training epochs is set to be 50. Different initial global learning rates are set for different CNNs. They are separately 0.001, 0.01, 0.001, and 0.05 for CaffeNet, GoogLeNet, VGG16, and ResNet101. After half of total training epochs, the initial global learning rate will drop by 10. We validate the convergence property of CNN on the training set and measure the accuracy on the testing set after every 5 iterations. The learning curves for CaffeNet fine-tuned on VMCLCars dataset are illustrated in Fig. 8. It can be seen that, during the fine-tuning procedure, CNN model is becoming more and more discriminative for vehicle recognition. Moreover, benefiting from the pre-training and fine-tuning strategies, our training speed is much quick. With a computer containing two GeForce GTX TITAN X GPUs, fine-tuning CaffeNet only takes 15 min.

Additionally, the input image is enlarged to two times, i.e. $448 \times 448 \times 3$ ($454 \times 454 \times 3$ for CaffeNet), when we employ fine-tuned CNN to extract features. This skill does not need to re-train CNN model, and can improve performance by increasing detail information. Using an enlarged input image and its features extracted from fine-tuned CaffeNet, our proposed GRPSP method only takes 0.04 s by averaging over 200 images. This is realized by a Intel(R) Core(TM) i7-6800K CPU and RAM with total memory of 16 GB.

4.3. Effects of different parameters

4.3.1. Feature extraction

In a fine-tuned CNN model, we need to select a layer that can generate the discriminative features and retain sufficient position information. Therefore only the non-fully-connected layers can satisfy the requirement. We empirically pick the last convolution or max-pooling layer that can output feature map of size greater than

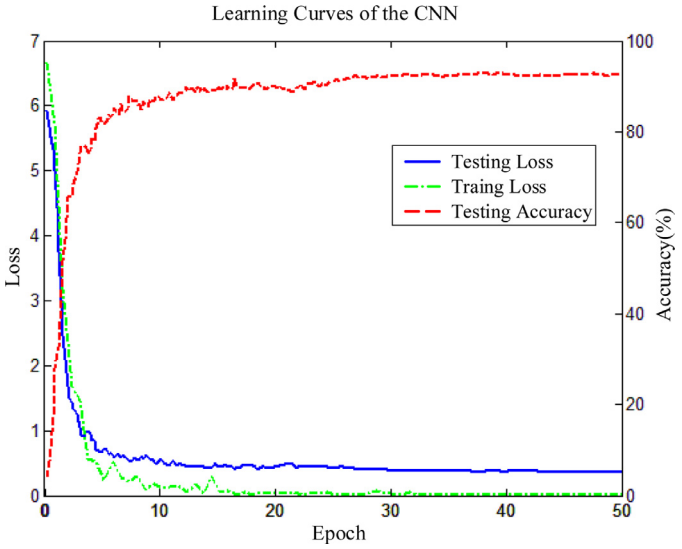


Fig. 8. Learning curves of CaffeNet fine-tuned on VMCLCars dataset. The blue solid line stands for the testing loss. The green dash-dot line represents the training loss. The red dashed line is the testing accuracy.

1, such as *pool5* layer for CaffeNet and VGG16, *inception_5b/output* layer for GoogLeNet, and *res5c_relu* layer for ResNet101. To prove the effectiveness of selected layer, we show the contrast result of different layers from CaffeNet on VMCLCars dataset in Table 1. It reveals that our choice is reasonable.

4.3.2. Global relative position space discretization

When we discretize the global relative position space for estimation of position probability, the number of discrete representatives which is namely the cluster number should be predetermined. To find the most proper cluster number for Kmeans++ algorithm, different numbers are practiced. Since extracted feature map of different CNNs has similar size (13×13 or 14×14), we use CaffeNet as an example to find the most proper number and share with other CNNs. In absolute position space of feature maps from CaffeNet, the total number of pixel coordinate is 13×13, i.e. 169. In view of this, we test several equidistant numbers from 90 to 150. Their corresponding results on VMCLCars dataset are illustrated in Table 2. It can be seen that the cluster number of 110, which is about the two-thirds of total pixel coordinate number in absolute position space, achieves the optimum result and is used in our method.

4.3.3. SVM learning

After global relative position space based pooling, the output representation vectors with fixed length of all training images are employed to train a linear SVM classifier. We take advantage of the frequently-used LIBLINEAR library [54] for SVM learning. Thereinto, the type of solver is set to be 1. The parameter μ in Eq. (12) should be tuned through cross validation for preventing the over-fitting problem. In VMCLCars dataset, the 2/7 of all the images for training final SVM model are randomly selected as validation set. The results of distinct μ values for proposed CaffeNet-GRPSP model on

Table 2
The average accuracy and overall accuracy with differnt cluster numbers in Kmeans++.

Cluster Number	90	100	110	120	130	140	150
Avr_Accuracy	95.81%	96.15%	**96.37%**	96.08%	95.63%	95.18%	94.66%
Ovr_Accuracy	96.20%	96.57%	**96.75%**	96.29%	95.91%	95.44%	95.09%

Table 3
The average accuracy and overall accuracy with differnt μ in SVM.

μ	0.6	1	1.4	1.8	2.2	2.6	3	3.4	3.8	4.2	4.6	5
Avr_Accuracy	94.10%	94.10%	94.14%	**94.31**%	94.30%	94.02%	94.02%	93.98%	93.98%	93.98%	93.98%	93.98%
Ovr_Accuracy	94.60%	94.60%	94.67%	**94.96**%	94.80%	94.46%	94.46%	94.40%	94.40%	94.40%	94.40%	94.40%

Table 4
Comparisons of our methods with other works on CompCars dataset.

Type	Methods	Avr_Accuracy	Ovr_Accuracy
Baselines	CaffeNet	94.81%	96.80%
	GoogLeNet	97.26%	98.95%
	VGG16	97.14%	98.77%
	ResNet101	98.57%	98.99%
Previous	Huang et al. [55]	93.82%	96.03%
	Hsieh et al. [9]	50.69%	55.61%
	Siddiqui et al. [7]	80.60%	84.35%
	Zhang [31]	82.58%	85.69%
	Biglari et al. [30]	95.55%	97.52%
	Fang et al. [22]	98.29%	98.63%
Ours	CaffeNet-GRPSP	98.98%	99.21%
	GoogLeNet-GRPSP	99.04%	99.30%
	VGG16-GRPSP	98.99%	99.25%
	ResNet101-GRPSP	**99.29%**	**99.39%**

Table 5
Comparisons of our methods with other works on VMCLCars dataset.

Type	Methods	Avr_Accuracy	Ovr_Accuracy
Baselines	CaffeNet	91.89%	93.08%
	GoogLeNet	94.62%	96.33%
	VGG16	94.47%	96.17%
	ResNet101	96.14%	96.58%
Previous	Huang et al. [55]	75.87%	79.80%
	Hsieh et al. [9]	49.96%	54.32%
	Siddiqui et al. [7]	64.85%	69.47%
	Zhang [31]	80.52%	83.58%
	Biglari et al. [30]	93.12%	94.85%
	Fang et al. [22]	94.69%	95.29%
Ours	CaffeNet-GRPSP	96.37%	96.75%
	GoogLeNet-GRPSP	96.58%	96.89%
	VGG16-GRPSP	96.38%	96.76%
	ResNet101-GRPSP	**97.07%**	**97.36**%

validation set of VMCLCars dataset are compared and shown in Table 3. Obviously, from 0.6 to 5, the value of 1.8 makes the best performance. We use it to train final SVM classifier for all proposed models combined with different CNNs on two datasets.

4.4. Comparison with state-of-the-arts

4.4.1. Comparison methods

We first directly use the four types of common CNNs described in Section 4.1 as baselines, including CaffeNet, GoogLeNet, VGG16, and ResNet101. Each of them can be combined with our proposed method. Then from previous works for fine-grained vehicle recognition, we pick out several recent and competitive ones. They are separately SIFT combined with LLC and SPM [55], symmetrical SURF descriptors [9], bag of SURF features for real-time vehicle recognition [7], two-stage cascade classifier ensemble [31], cascaded part-based system [30], and coarse-to-fine CNN [22]. Thereinto, Siddiqui et al. [7] discussed different schemes for dictionary building and multi-class classification. We implement it with the single-dictionary scheme and the single multi-class SVM classifier since the accuracy is higher.

4.4.2. Results and analyses

The two datasets described in Section 4.2.1, which are CompCars and VMCLCars, are adopted to compare our method with other works. Comparison results are illustrated in Tables 4 and 5, respectively. In these two tables, "Avr_Accuracy" and "Ovr_Accuracy" are computed by Eq. (13). Since same CompCars dataset and metrics are adopted in [22,30], we directly copy their results into Table 4. The other results are all obtained from our implementation. The majority of methods we implement take the SVM as classification algorithm. To make the comparisons sufficiently fair, we set the regularization parameter μ in Eq. (12) through cross validation for all methods, including ours. Other parameters for different methods are set according to their corresponding papers. From the experiment results on two datasets, we can observe that our method has achieved the state-of-the-art performance in terms of both the average accuracy and overall accuracy.

For all methods to be compared, the experiment results on CompCars dataset are better than that on VMCLCars dataset. It reveals that our VMCLCars dataset from web is more challenging than the public CompCars dataset from surveillance. On the two datasets, all our methods have effectively improved various CNNs in baselines. Among these various CNNs, ResNet101 performs best. Similarly, our method based on ResNet101, i.e. ResNet101-GRPSP, is state-of-the-art on both two datasets. Besides, during multiple random splits for our self-building VMCLCars dataset, ResNet101-GRPSP achieves average accuracies that change between 97.01% and 97.15%. The small amplitude demonstrates the reliability of our proposed model.

To further prove the effectiveness of our proposed ResNet101-GRPSP model, the confusion matrix is adopted to illustrate the accuracy in each class. We show confusion matrices for ResNet101 model and proposed ResNet101-GRPSP model on CompCars dataset in Fig. 9a and 9b, respectively. We can see that the number of samples which are misclassified is largely reduced by our proposed method. The confusion matrices of two same models on VMCLCars dataset are shown in Fig. 10a and 10b. Since the less testing images and more classes of VMCLCars dataset, the samples which are misclassified in Fig. 10 scatter widely, and are not as clear as in Fig. 9. However, it still can be seen that the number of wrongly classified samples is decreased when we enlarge the figure. Therefore, our proposed ResNet101-GRPSP model can improve original ResNet101 model effectively.

Comparing with previous methods, we summarize several points. Firstly, method [55] with SIFT as feature extraction algorithm is superior to methods [7,9] which are both based on

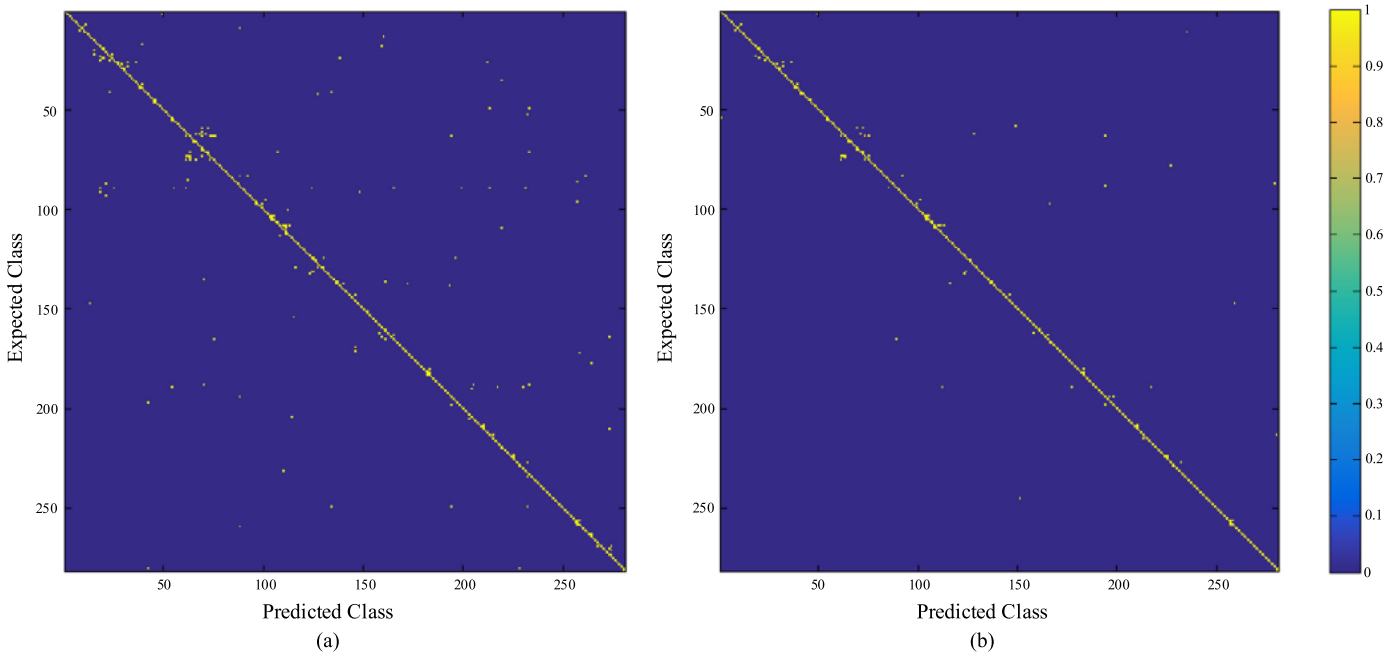


Fig. 9. Comparison of confusion matrices for ResNet101 and ResNet101-GRPSP models on CompCars dataset. (a) is for ResNet101 model, and (b) is for ResNet101-GRPSP model.

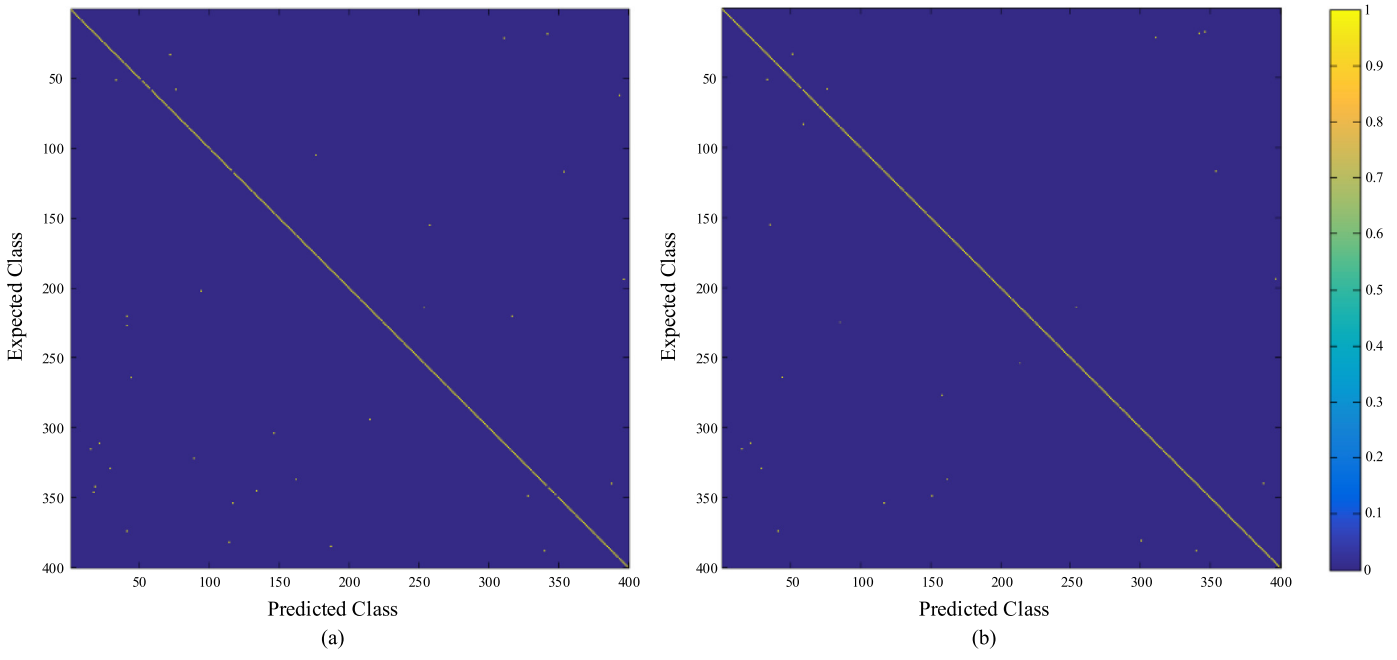


Fig. 10. Comparison of confusion matrices for ResNet101 and ResNet101-GRPSP models on VMCLCars dataset. (a) is for ResNet101 model, and (b) is for ResNet101-GRPSP model. (Due to the less testing images and more classes of VMCLCars dataset, the samples which are misclassified in Fig. 10 scatter widely and are not as clear as in Fig. 9.)

SURF features. It is in accordance with the conclusion in [9]. Secondly, method of [31] shows similar performances on two different datasets, testifying the stability of this method. Thirdly, both [22] and [30] aim to find discriminative parts automatically, but [22] wins obviously for more semantical features. Fourthly, both [22] and our method are based upon convolutional neural networks to extract features, and obtain better results than other works, even on the small VMCLCars dataset which owns extremely few training samples. It is because there exists the pre-processing of data augmentation [43], which consists of generating image translations and horizontal reflections. Lastly, our method defeats [22] with larger advantage on VMCLCars dataset than on CompCars dataset. The reason is that the angle of pitch for image in VMCLCars dataset from web varies more widely, which leads to more instances of wrong localization for [22]. Moreover, the much fewer training samples in VMCLCars dataset could make [22] perform worse. On the contrary, the strength of our algorithm based on relative position is highlighted on this challenging dataset.

5. Conclusion

In this paper, we proposed a novel pooling method based upon the global relative position space, and applied it on the convolutional features extracted by CNN for fine-grained vehicle

recognition. This technique can effectively align discriminative features and reduce intra-class variations, which are introduced by unstable absolute position changes of features. Therefore, compared with several state-of-the-art methods, our method performs the best on both the public CompCars dataset from surveillance and the self-building challenging VMCLCars dataset from web.

In future, we will explore or design more effective CNN architecture to provide more sufficient position information for discriminative convolutional features. Since the architecture of current CNN for classification only has small feature map size in posterior layer, the considerable loss of position information has been made. Hence we need more sufficient position information to further improve the accuracy of fine-grained vehicle recognition.

Declarations of interest

None.

Acknowledgment

This work was supported by the National Natural Science Foundation of China under Grant No. 61425013.

References

[1] B. Tian, B.T. Morris, M. Tang, Y. Liu, Y. Yao, C. Gou, D. Shen, S. Tang, Hierarchical and networked vehicle surveillance in ITS: a survey, IEEE Trans. Intell. Transp. Syst. 16 (2) (2015) 557–580.
[2] A. Boukerche, A.J. Siddiqui, A. Mammeri, Automated vehicle detection and classification: models, methods, and techniques, ACM Comput. Surv. 50 (5) (2017) 1–39.
[3] G. Pearce, N. Pears, Automatic make and model recognition from frontal images of cars, in: Proceedings of the IEEE International Conference on Advanced Video Signal Surveillance, 2011, pp. 373–378.
[4] Y. Tang, C. Zhang, R. Gu, P. Li, B. Yang, Vehicle detection and recognition for intelligent traffic surveillance system, Multimed. Tools Appl. 76 (4) (2017) 5817–5832.
[5] D. Llorca, D. Colás, I. Daza, I. Parra, M. Sotelo, Vehicle model recognition using geometry and appearance of car emblems from rear view images, in: Proceedings of the IEEE International Conference on Intelligent Transportation Systems, 2014, pp. 3094–3099.
[6] J. Sochor, A. Herout, J. Havel, Boxcars: 3D boxes as CNN input for improved fine-grained vehicle recognition, in: Proceedings of the IEEE IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 3006–3015.
[7] A.J. Siddiqui, A. Mammeri, A. Boukerche, Real-time vehicle make and model recognition based on a bag of SURF features, IEEE Trans. Intell. Transp. Syst. 17 (11) (2016) 3205–3219.
[8] L.C. Chen, J.W. Hsieh, Y. Yan, D.Y. Chen, Vehicle make and model recognition using sparse representation and symmetrical SURFs, Pattern Recognit. 48 (6) (2015) 1979–1998.
[9] J.W. Hsieh, L.C. Chen, D.Y. Chen, Symmetrical SURF and its applications to vehicle detection and vehicle make and model recognition, IEEE Trans. Intell. Transp. Syst. 15 (1) (2014) 6–20.
[10] P. Badura, M. Skotnicka, Automatic car make recognition in low-quality images, in: E. Pitka, J. Kawa, W. Wieclawek (Eds.), Information Technologies in Biomedicine, Advances in Intelligent Systems and Computing, 3, Springer-Verlag, 2014, pp. 235–246.
[11] R. Baran, A. Glowacz, A. Matiolanski, The efficient real- and non-real-time make and model recognition of cars, Multimed. Tools Appl. 74 (12) (2015) 4269–4288.
[12] M. Fraz, E.A. Edirisinghe, M.S. Sarfraz, Mid-level-representation based lexicon for vehicle make and model recognition, in: Proceedings of the IEEE International Conference on Pattern Recognition, 2014, pp. 393–398.
[13] S. Yu, Y. Wu, W. Li, Z. Song, W. Zeng, A model for fine-grained vehicle classification based on deep learning, Neurocomputing 257 (27) (2017) 97–103.
[14] Y. Gao, H.J. Lee, Local tiled deep networks for recognition of vehicle make and model, Sensors 16 (2) (2016).
[15] R. Girshick, J. Donahue, T. Darrell, J. Malik, Rich feature hierarchies for accurate object detection and semantic segmentation, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2014, pp. 580–587.
[16] F. Liu, G. Lin, C. Shen, CRF Learning with CNN features for image segmentation, Pattern Recognit. 48 (10) (2015) 2983–2992.
[17] J. Long, E. Shelhamer, T. Darrell, Fully convolutional networks for semantic segmentation, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015, pp. 3431–3440.
[18] L. Yang, P. Luo, C.C. Loy, X. Tang, A large-scale car dataset for fine-grained categorization and verification, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015, pp. 3973–3981.
[19] B. Hu, J.H. Lai, C.C. Guo, Location-aware fine-grained vehicle type recognition using multi-task deep networks, Neurocomputing 243 (21) (2017) 60–68.
[20] K. Grauman, T. Darrell, The pyramid match kernel: Discriminative classification with sets of image features, in: Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2005, pp. 1458–1465.
[21] S. Lazebnik, C. Schmid, J. Ponce, Beyond bags of features: spatial pyramid matching for recognizing natural scene categories, in: Proceedings of the IEEE Computer Society Conference on Computer Vision Pattern Recognition, 2, 2006, pp. 2169–2178.
[22] J. Fang, Y. Zhou, Y. Yu, S. Du, Fine-grained vehicle model recognition using a coarse-to-fine convolutional neural network architecture, IEEE Trans. Intell. Transp. Syst. 18 (7) (2017) 1782–1792.
[23] L. Liu, C. Shen, A.V.D. Hengel, The treasure beneath convolutional layers: Cross-convolutional-layer pooling for image classification, in: Proceedings of the IEEE Conference on Computer Vision Pattern Recognition (CVPR), 2015, pp. 4749–4757.
[24] C.J.C. Burges, A tutorial on support vector machines for pattern recognition, Data Min. Knowl. Discov. 2 (2) (1998) 121–167.
[25] X.X. Niu, C.Y. Suen, A novel hybrid CNN-SVM classifier for recognizing handwritten digits, Pattern Recognit. 45 (4) (2012) 1318–1325.
[26] B. Zhang, C. Zhao, Classification of vehicle make by combined features and random subspace ensemble, in: Proceedings of the IEEE International Conference Image Graphics, 2011, pp. 920–925.
[27] H. Huang, Q. Zhao, Y. Jia, S. Tang, A 2DLDA based algorithm for real time vehicle type recognition, in: Proceedings of the IEEE Conference on Intelligent Transportation Systems, 2008, pp. 298–303.
[28] A. Psyllos, C.N. Anagnostopoulos, E. Kayafas, Vehicle model recognition from frontal view image measurements, Comput. Stand. Interfaces 33 (2) (2011) 142–151.
[29] S. Saravi, E.A. Edirisinghe, Vehicle make and model recognition in CCTV footage, in: Proceedings of the IEEE International Conference on Communication and Signal Processing, 2013, pp. 1–6.
[30] M. Biglari, A. Soleimani, H. Hassanpour, A cascaded part-based system for fine-grained vehicle classification, IEEE Trans. Intell. Transp. Syst. 19 (1) (2018) 273–283.
[31] B. Zhang, Reliable classification of vehicle types based on cascade classifier ensembles, IEEE Trans. Intell. Transp. Syst. 14 (1) (2013) 322–332.
[32] P. Tang, X. Wang, Z. Huang, X. Bai, W. Liu, Deep patch learning for weakly supervised object classification and discovery, Pattern Recognit. 71 (2017) 446–459.
[33] K. He, X. Zhang, S. Ren, J. Sun, Spatial pyramid pooling in deep convolutional networks for visual recognition, IEEE Trans. Pattern Anal. Mach. Intell. 37 (9) (2015) 1904–1916.
[34] Y. Zhou, L. Liu, L. Shao, M. Mellor, Dave: A unified framework for fast vehicle detection and annotation, in: Proceedings of the European Conference on Computer Vision, 2016, pp. 278–293.
[35] F. Perronnin, J. Sánchez, T. Mensink, Improving the fisher kernel for large-scale image classification, in: Proceedings of the European Conference on Computer Vision, 2010, pp. 143–156.
[36] P. Tang, X. Wang, B. Shi, X. Bai, W. Liu, Z. Tu, Deep FisherNet for image classification, IEEE Trans. Neural Netw. Learn. Syst. 30 (7) (2019) 2244–2250.
[37] J. Wang, J. Yang, K. Yu, F. Lv, T. Huang, Y. Gong, Locality-constrained linear coding for image classification, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2010, pp. 3360–3367.
[38] J. Krause, M. Stark, J. Deng, F.F. Li, 3D object representations for fine-grained categorization, in: Proceedings of the IEEE International Conference on Computer Vision Workshops (ICCVW), 2013, pp. 554–561.
[39] Y. Peng, Y. Yan, W. Zhu, J. Zhao, Vehicle classification using sparse coding and spatial pyramid matching, in: Proceedings of the IEEE International Conference on Intelligent Transportation System (ITSC), 2014, pp. 259–263.
[40] W. Shen, C. Du, Y. Jiang, D. Zeng, Z. Zhang, Bag of shape features with a learned pooling function for shape recognition, Pattern Recognit. Lett. 106 (15) (2018) 33–40.
[41] N. Boonsim, S. Prakoonwit, Car make and model recognition under limited lighting conditions at night, Pattern Anal. Appl. 20 (4) (2016) 1195–1207.
[42] M. Liu, C. Yu, H. Ling, J. Lei, Hierarchical joint CNN-based models for fine-grained cars recognition, in: Proceedings of the IEEE International Conference on Cyber Security and Cloud Computing, 2016, pp. 337–347.
[43] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, in: Proceedings of the Advances in Neural Information Processing Systems, 2012, pp. 1097–1105.
[44] Y. Sun, X. Wang, X. Tang, Deep learning face representation from predicting 10,000 classes, in: Proceedings of the IEEE Conference on Computer Vision Pattern Recognition (CVPR), 2014, pp. 1891–1898.
[45] Z. Zhu, P. Luo, X. Wang, X. Tang, Multi-view perceptron: a deep model for learning face identity and view representations, in: Proceedings of the International Conference on Neural Information Processing Systems, 2014, pp. 217–225.
[46] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, T. Darrell, Caffe: convolutional architecture for fast feature embedding, in: Proceedings of the ACM International Conference on Multimedia, 2014, pp. 675–678.
[47] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, A. Rabinovich, Going deeper with convolutions, in: Proceedings of the IEEE Conference on Computer Vision Pattern Recognition (CVPR), 2015, pp. 1–9.

[48] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, CoRR (2014). arXiv preprint arXiv: 1409.1556.
[49] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision Pattern Recognition (CVPR), 2016, pp. 770–778.
[50] T.Y. Lin, A. Roychowdhury, S. Maji, Bilinear convolutional neural networks for fine-grained visual recognition, IEEE Trans. Pattern Anal. Mach. Intell. PP (99) (2017) 1.
[51] J. Fu, H. Zheng, T. Mei, Look closer to see better: recurrent attention convolutional neural network for fine-grained image recognition, in: Proceedings of the IEEE Conference on Computer Vision Pattern Recognition (CVPR), 2017, pp. 4476–4484.
[52] D. Arthur, S. Vassilvitskii, K-means++: the advantages of careful seeding, in: Proceedings of the ACM-SIAM Symposium on Discrete Algorithms, 2007, pp. 1027–1035.
[53] J. Deng, W. Dong, R. Socher, L.J. Li, K. Li, F.F. Li, ImageNet: a large-scale hierarchical image database, in: Proceedings of the IEEE Conference on Computer Vision Pattern Recognition (CVPR), 2009, pp. 248–255.
[54] R.E. Fan, K.W. Chang, C.J. Hsieh, X.R. Wang, C.J. Lin, LIBLINEAR: a library for large linear classification, J. Mach. Learn. Res. 9 (2008) 1871–1874.
[55] Y. Huang, Z. Wu, L. Wang, T. Tan, Feature coding in image classification: a comprehensive study, IEEE Trans. Pattern Anal. Mach. Intell. 36 (3) (2014) 493–506.

Ye Xiang is currently working toward the Ph.D. degree in the School of Computer Science and Technology, Beijing Institute of Technology, China. She received the B.S. degree from University of Science and Technology Beijing in 2012. Her main research interests include computer vision and machine learning.

Ying Fu received the B.S. degree in Electronic Engineering from Xidian University in 2009, the M.S. degree in Automation from Tsinghua University in 2012, and the Ph.D. degree in information science and technology from the University of Tokyo in 2015. She is currently an Associate Professor with the School of Computer Science and Technology, Beijing Institute of Technology. Her research interests include physics-based vision, image processing, computational photography, and machine learning.

Hua Huang received the B.S. and Ph.D. degrees from Xi'an Jiaotong University, China, in 1996 and 2006, respectively. He is currently a professor at the School of Computer Science and Technology, Beijing Institute of Technology, China. He is also an adjunct professor at the School of Electronics and Information Engineering, Xi'an Jiaotong University, China. His main research interests include image and ideo processing and computer graphics. He is an Associate Editor of IEEE Transactions on Intelligent Transportation Systems. He is also an IEEE member.